

Project Documentation: Biker Gallery

Bike Rental System

1. Modules Description

The system is divided into simple modules to manage bike rentals and user accounts.

User Module:

Allows users to create an account (Register).

Allows users to log in securely.

Includes KYC verification (PAN Card & Driving License scan).

Bike Catalog Module:

Displays available bikes (125cc, 200cc, Premium).

Shows bike details (Engine, Mileage, Price).

Filters bikes by category.

Booking Module:

Allows users to select a bike, date, and duration (hours).

Automatically calculates the total cost (Rate + GST).

Checks bike availability in real-time.

KYC Verification Module:

Uses OCR (Optical Character Recognition) to scan uploaded documents.

Auto-fills user details (Name, DOB, ID Number) from scanned images.

Admin Module (Implicit):

Manages bike availability (Available/Unavailable).

Views booking requests.

Handles user queries from the Contact form.

2. UI Form Design

The user interface is designed to be clean and mobile-friendly.

Login Page (login.html):

Fields: Username, Password.

Features: Show/Hide password toggle, "Forgot Password" link, Redirect to Register.

Registration Page (register.html):

Personal Info: Username, Full Name, Date of Birth, Email, Phone, Password.

Document Upload: PAN Card Image, Driving License Image.

Features: Real-time validation, OCR Scan status indicator.

Home Page (home.html):

Content: Welcome message, Quick stats (Bikes available, Price), Navigation links.

Menu Page (menu.html):

Layout: Grid of bike cards.

Filters: Buttons for All, 125cc, 200cc, Premium.

Card Info: Bike Name, Price/hr, "Book Now" button.

Booking Page (book.html):

Selection: Selected Bike Name, Rate.

Form: Full Name, Phone, Pickup Date, Duration (Hours).

Summary: Base Cost, GST (18%), Total Cost.

Contact Page (contact.html):

Fields: Name, Phone, Message.

Info: Address, Phone Number, Working Hours.

3. Database Table Structure (MySQL)

The following tables are required to store data securely.

Table 1: users

Field Name	Type	Description
`id`	INT (PK)	Unique User ID
`username`	VARCHAR(25)	Login username
`password`	VARCHAR(64)	SHA-256 Hashed Password
`full_name`	VARCHAR(35)	User's Full Name
`dob`	DATE	Date of Birth
`email`	VARCHAR(64)	Email Address
`phone`	VARCHAR(10)	10-Digit Phone Number
`pan_data`	TEXT	Scanned PAN Details
`dl_data`	TEXT	Scanned License Details

Table 2: bikes

Field Name	Type	Description
`id`	INT (PK)	Unique Bike ID
`bike_name`	VARCHAR(50)	e.g., Honda SP 125
`category`	VARCHAR(20)	125cc, 200cc, Premium
`rate_per_hr`	DECIMAL(10,2)	Price per hour
`status`	VARCHAR(10)	'available' or 'booked'

Table 3: bookings

Field Name	Type	Description
`id`	INT (PK)	Unique Booking ID
`user_id`	INT (FK)	Reference to `users` table
`bike_name`	VARCHAR(50)	Name of booked bike
`booking_date`	DATE	Pickup Date
`hours`	INT	Duration in hours
`total_amount`	DECIMAL(10,2)	Final Price (incl. GST)
`status`	VARCHAR(20)	Confirmed, Completed, Cancelled

Table 4: contact_messages

Field Name	Type	Description
`id`	INT (PK)	Message ID
`name`	VARCHAR(50)	Sender Name
`phone`	VARCHAR(10)	Sender Phone
`message`	TEXT	User Message

4. Proposed System Architecture

The system follows a 3-Tier Architecture.

Presentation Layer (Frontend):

Technologies: HTML5, CSS3, JavaScript .

Libraries: Tesseract.js (for OCR scanning).

Responsibility: Displays forms, validates user input, calculates prices on the client side.

Application Layer (Backend):

Technology: PHP.

Files: register.php, login.php, api_book.php, api_availability.php.

Responsibility: Processes forms, hashes passwords (SHA-256), manages database connections, handles file uploads.

Data Layer (Database):

Technology: MySQL.

Responsibility: Stores user credentials, bike details, and booking records securely.

Security Note: Passwords are hashed using SHA-256 in JavaScript before being sent to the server to ensure security during transmission.

5. Flow Diagram (Step-by-Step Working)

User Registration Flow:

User opens Register Page.

User fills in personal details.

User uploads PAN and DL images.

Scanner.js runs OCR to extract text and auto-fills fields.

User clicks "Complete Registration".

JavaScript validates inputs (Age > 18, Phone = 10 digits).

Password is hashed.

Data sent to register.php -> Saved in MySQL.

Redirect to Home Page.

Booking Flow:

User opens Menu Page -> Selects a Bike.

Redirects to Book Page with bike details.

System checks Availability API.

User enters Date and Hours.

System calculates Total ($\text{Rate} \times \text{Hours} + \text{GST}$).

User clicks "Confirm Booking".

Data sent to api_book.php -> Saved in MySQL.

Show Success Message.

6. UML Diagrams

A. Use Case Diagram

Actors: User, Admin.

User Cases:

Register Account.

Login.

Upload KYC Documents.

View Bike Menu.

Book a Ride.

Contact Support.

Admin Cases:

Manage Bike Availability.

View Bookings.

Reply to Contact Messages.

B. Class Diagram

Class: User

Attributes: username, password, phone, kyc_status

Methods: register(), login(), updateProfile()

Class: Bike

Attributes: name, category, price, availability

Methods: getDetails(), checkStatus()

Class: Booking

Attributes: date, hours, total_cost, status

Methods: createBooking(), cancelBooking()

Class: KYCScanner

Attributes: pan_image, dl_image

Methods: scanDocument(), extractData()

7. ER Diagram (Entity Relationship)

Entities:

USER

BIKE

BOOKING

Relationships:

USER makes BOOKING (One-to-Many).

One user can make many bookings.

BIKE is included in BOOKING (One-to-Many).

One bike model can be booked many times.

USER uploads KYC (One-to-One).

One user has one set of KYC documents.